

Abstract

This proposal outlines a final project for the NLP laboratory course, focused on building and comparing four core sequence-modeling architectures taught in the course: Artificial Neural Networks (ANN), Recurrent Neural Networks (RNN), Long Short-Term Memory networks (LSTM), and Transformer models with self-attention. All models are implemented and trained entirely from scratch in PyTorch, without the use of any pretrained weights, pretrained tokenizers, or pretrained base models. The architectures are evaluated on a small, custom-built dataset of simple reasoning problems expressed in English, Bangla, Banglish, and Bangla–English code-switched form, with the objective of observing how each architecture handles code-switched input relative to single-language input. The project is intended purely as an implementation and comparison exercise consistent with the techniques covered in the course, rather than as an open-ended research study.

Keywords: Bangla–English Code-Switching, ANN, RNN, LSTM, Attention, Transformer, PyTorch, From-Scratch Implementation, NLP Laboratory Project.

1. Project Type

This is an implementation-oriented final project for the NLP laboratory course. The project requires no external research contribution and is scoped strictly to the architectures and techniques covered in the course curriculum: Artificial Neural Networks, Recurrent Neural Networks, Long Short-Term Memory networks, Attention mechanisms, and Transformer models.

2. Introduction

In Bangladesh, everyday communication frequently combines Bangla and English within a single sentence or utterance, a phenomenon commonly referred to as code-switching. This project uses Bangla–English code-switching as a practical and linguistically relevant setting in which to implement, train, and compare the sequence-modeling architectures introduced in this course.

All models in this project are implemented from first principles in PyTorch and trained from random initialization on a small, custom-built dataset. No pretrained or pre-built base model of any kind is used at any stage of the project, in accordance with the course requirement that all architectures be built and understood from scratch.

3. Project Objective

The primary objective of this project is to implement each architecture covered in the course and evaluate its ability to solve simple reasoning problems when the input text is written in English, Bangla, Banglish, or a code-switched mixture of Bangla and English.

The specific objectives of the project are as follows:

1. To implement an ANN, an RNN, an LSTM, and a Transformer model from scratch in PyTorch, without the use of any pretrained weights, pretrained tokenizers, or external base models.
2. To construct a small, custom dataset of reasoning problems in English, Bangla, Banglish, and Bangla–English code-switched form.
3. To train each model independently on an identical dataset and identical data splits, so that results are directly comparable.
4. To evaluate whether code-switched input is more difficult for these models to process correctly than single-language input.
5. To present the results as a clear, architecture-by-architecture comparison, supported by a brief qualitative error analysis.

4. Scope and Constraints

The following constraints define the scope of the project and are treated as binding requirements throughout implementation:

- **No pretrained models.** All models are implemented and trained from scratch in PyTorch. No pretrained language models, pretrained embeddings (e.g., word2vec, GloVe, fastText), or pretrained tokenizers are used.
- **No external LLM APIs for the models under evaluation.** A large language model may optionally be used only as an auxiliary tool for generating dataset templates, and not as a component whose performance is being measured.
- **Small, synthetic, template-based dataset.** Given the limited compute and time available for training models from random initialization, the reasoning tasks are deliberately kept simple and are generated using controlled templates.
- **Custom tokenizer.** A tokenizer for the mixed Bangla–English vocabulary is implemented manually, rather than using any pretrained subword tokenizer.

5. Dataset Construction

A small, custom dataset of simple reasoning problems is generated using controlled templates, in four parallel linguistic forms:

- **English** — e.g., *“If Rahim has 5 apples and gives away 2, how many does he have left?”*
- **Bangla** — the same problem expressed fully in the Bangla script.
- **Banglish** — the same problem expressed in Bangla, written using the Roman (English) alphabet.
- **Code-switched** — the same problem with Bangla and English mixed within a single sentence.

The dataset covers the following reasoning categories:

- Simple arithmetic word problems
- Basic logical statements (conditional, if-then reasoning)
- Simple sequence and pattern completion
- Short instruction-following prompts

Each template is instantiated multiple times with varying names, quantities, and objects, in order to produce a sufficiently large and varied set of training, validation, and test examples.

6. Model Architectures

All models are implemented from scratch in PyTorch and trained on identical data splits, ensuring that the resulting comparison is fair and controlled.

1. **ANN (Baseline)**. A feed-forward network operating over a bag-of-words or fixed-window input representation. This model is expected to perform poorly on tasks requiring sequence order or long-range dependency, and serves as a lower-bound baseline.
2. **RNN**. A basic recurrent network that processes the input sequence token by token, maintaining a hidden state across time steps.
3. **LSTM**. A Long Short-Term Memory network, used to evaluate whether improved long-range memory retention aids performance on code-switched input.

4. **Attention-Augmented Model.** An LSTM (or RNN) combined with a manually implemented attention mechanism, allowing the model to selectively weight relevant portions of the input sequence.
5. **Transformer.** A small Transformer encoder, consisting of a limited number of self-attention layers and positional encodings, implemented entirely from scratch without reliance on any pretrained transformer library.

7. Evaluation Plan

- Each model is trained and evaluated using identical training, validation, and test splits.
- Accuracy, or another task-appropriate metric, is measured separately for English, Bangla, Banglish, and code-switched test examples.
- Results are compared across all five models to identify which architecture demonstrates the greatest robustness to code-switched input.
- A brief qualitative error analysis is conducted, presenting representative failure cases for each model.

8. Expected Deliverables

- A small, custom Bangla–English code-switched reasoning dataset, including the templates and scripts used to generate it.
- PyTorch implementations of the ANN, RNN, LSTM, Attention-augmented, and Transformer models, built entirely from scratch.
- Training and evaluation code, with results logged for each model and each language condition.
- A comparative table and accompanying chart summarizing accuracy across all models and language conditions.
- A final written report summarizing the project setup, methodology, results, and observations.

9. Tentative Timeline

Week(s)	Activity
Week 1	Dataset template design and generation
Week 2	Custom tokenizer implementation and data pipeline setup
Week 3	Implementation and training of the ANN and RNN baselines
Week 4	Implementation and training of the LSTM and attention-augmented models
Week 5	Implementation and training of the Transformer model
Week 6	Evaluation, comparative analysis, error analysis, and final report preparation

10. Tools and Technologies

- **Framework:** PyTorch (all models implemented from scratch)
- **Programming Language:** Python
- **Data Handling:** Custom dataset generation scripts and a manually implemented tokenizer
- **Evaluation:** Standard accuracy and task-specific metrics, with results visualized using Matplotlib

11. Conclusion

This project provides a structured, hands-on comparison of the core sequence-modeling architectures taught in the NLP laboratory course, using Bangla–English code-switching as a practical evaluation setting. By implementing every model from scratch in PyTorch and evaluating them under controlled, identical conditions, the project is expected to yield a clear and instructive comparison of how ANN, RNN, LSTM, Attention-augmented, and Transformer architectures differ in their ability to process linguistically mixed input.